

Artificial Neural Networks

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

Background

The Perceptron

Exclusive OR?

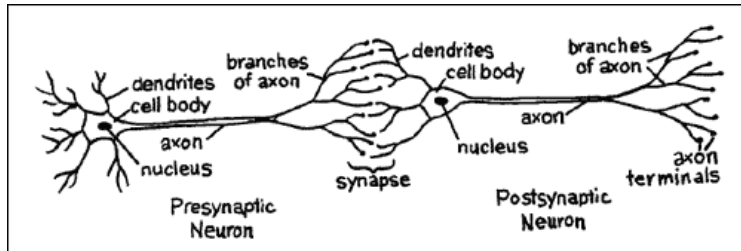
Activation Functions

Multilayer Neural Networks

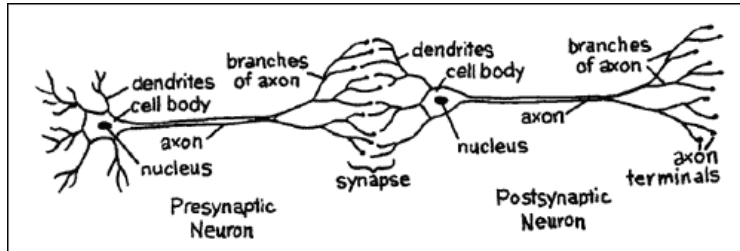
Back-propagation Learning Algorithm

- ▶ Artificial neural networks (ANNs) are inspired from the human brain.
- ▶ Human brain consists of densely interconnected set of nerve cells called neurons (basic information-processing units).

- ▶ Each neuron has a very simple structure that consists of
 - a cell body called soma,
 - a number of fibres called dendrites, and
 - a single long fibre called axon.
- ▶ The neurons are connected with synapses.



- ▶ A neuron receives signals through dendrites.
- ▶ A signal is received in the form of chemical substances released from the synapses.
- ▶ This changes the electrical potential of the cell body.
- ▶ If the potential exceeds the threshold, an electrical pulse is sent through the axon then to the synapses to be sent to other neurons.



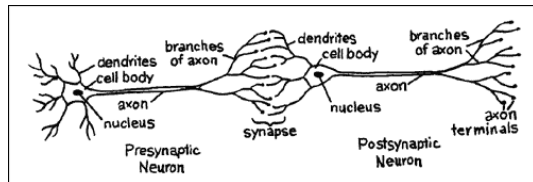
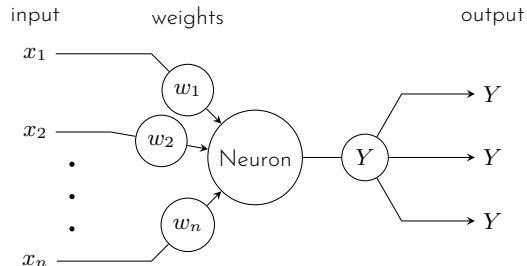
Plasticity of a biological neural networks

- ▶ While responding to stimulations, neurons demonstrate long-term changes in the strength of their connections.
- ▶ New connections with other neurons may be formed.
- ▶ This forms the basis for learning in the brain.
- ▶ Connections between neurons leading to the 'right answer' are strengthened while those leading to the 'wrong answer' weaken.
- ▶ As a result, neural networks have the ability to learn through experience.

Background

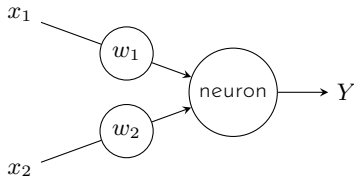
ANNs simulate biological neurons using artificial neurons.

Biological neuron	Artificial neuron
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight



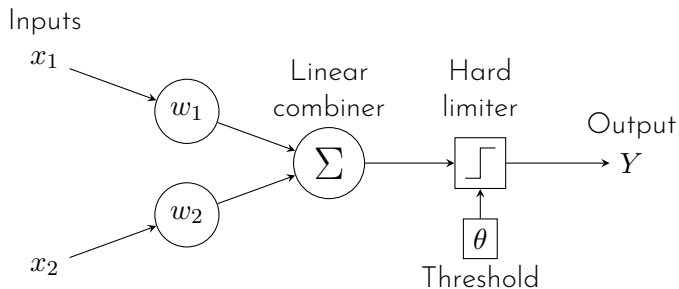
Simplest neural network

The simplest neural network is a perceptron.



How do neural network produce outputs?

Operation of Rosenblatt's perceptron



x_1	$w_1 x_1$	$w_1 x_1 + w_2 x_2$
x_2	$w_2 x_2$	

$$Y = \begin{cases} +1, & w_1 x_1 + w_2 x_2 - \theta \geq 0 \\ -1, & w_1 x_1 + w_2 x_2 - \theta < 0 \end{cases}$$

Rosenblatt's perceptron

The aim of the Rosenblatt's perceptron is to classify the inputs, i.e. to group the inputs into two classes.

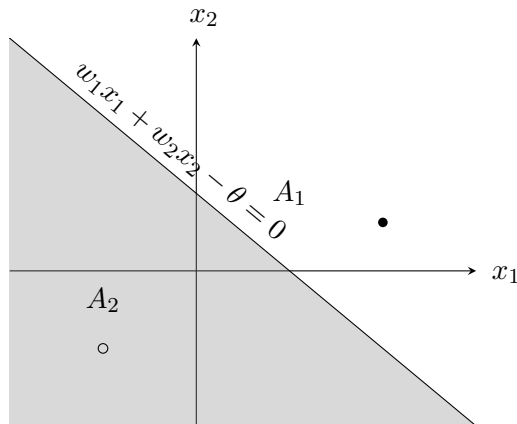
If $w_1 = 3$, $w_2 = 5$, and $\theta = 20$,

let's say $Y = +1$ is considered class A_1 and $Y = -1$ is A_2 ,

x_1	x_2	$w_1x_1 + w_2x_2 - \theta$	Y	class
1.25	5.24	9.95	+1	A_1
5.29	-100.5	-506.63	-1	A_2
0.01	-7.36	-56.77	-1	A_2
2.25	6.51	19.3	+1	A_1

Rosenblatt's perceptron

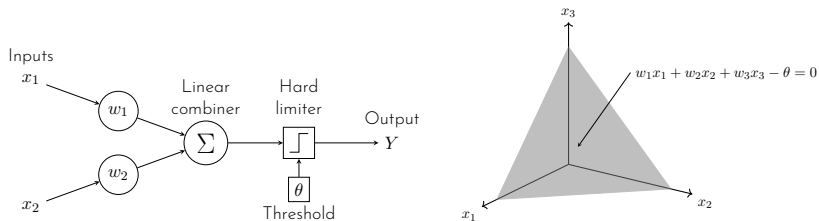
The Rosenblatt's perceptron is a classifier which divides the 2-dimensional space into two decision regions by the hyperplane defined by $w_1x_1 + w_2x_2 - \theta = 0$.



Perceptron with three inputs

With three inputs, the output is given by

$$Y = \begin{cases} +1, & w_1x_1 + w_2x_2 + w_3x_3 - \theta \geq 0 \\ -1, & w_1x_1 + w_2x_2 + w_3x_3 - \theta < 0 \end{cases}$$



Perceptron with multiple inputs

Therefore, the output Y of a multiple inputs $x_1, x_2, \dots, x_n$ can be obtained by

$$Y = \begin{cases} +1, & \sum_{i=1}^n w_i x_i - \theta \geq 0 \\ -1, & \sum_{i=1}^n w_i x_i - \theta < 0 \end{cases}$$

The hyperplane that separate the two classes is defined by the linearly separable function

$$\sum_{i=1}^n w_i x_i - \theta = 0$$

How do perceptron learn?

In the process of learning, a set of data (i.e. input and output pairs) is used to train the weights of the connections.

The weights are adjusted to reduce the difference between the actual and desired outputs of the perceptron.

Procedure in training a perceptron

Set initial weights, $w_1, w_2, \dots, w_n$ and threshold θ to random numbers in range $[-0.5, 0.5]$.

Repeat the process of

Activation - Weight training - Iteration

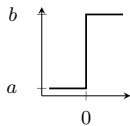
until termination (convergence)



Activate the perceptron to calculate the output of the perceptron using a step activation function.

$$Y = \text{step}\left(\sum_{i=0}^n w_i x_i - \theta\right)$$

$$\text{step}(x) = \begin{cases} a & x \geq 0 \\ b & x < 0 \end{cases}$$



Calculate error between desired output Y_d and the calculated output Y

$$e = Y_d - Y$$

Calculate weight (and θ) correction using the delta rule

$$\Delta w_i = \alpha \times x_i \times e \quad \Delta \theta = \alpha \times -1 \times e$$

(α is the learning rate, a positive constant less than 1)

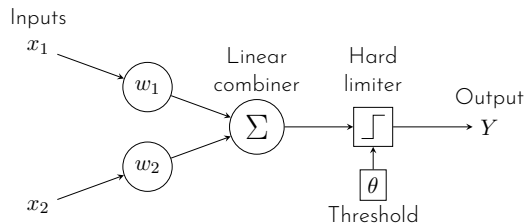
Calculate the new weights of the perceptron

$$w'_i = w_i + \Delta w_i \quad \theta' = \theta + \Delta \theta$$

Example 1

Learn the AND logical operation using a perceptron.

Inputs		AND
x_1	x_2	$x_1 \cap x_2$
0	0	0
0	1	0
1	0	0
1	1	1



Example 1

1. Initialisation - initialise w_1, w_2, θ to $[-0.5, 0.5]$, define α
2. Activation - calculate Y
3. Weight training - calculate $e, \Delta w_i, \Delta \theta, w'_i$, and θ'
4. Repeat

Sample of learning process for Example 1

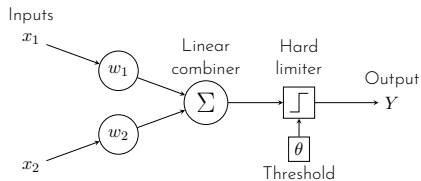
$$\alpha = 0.1$$

Epoch	Iteration	Inputs		Desired	Weights & Threshold			$\sum w_i x_i - \theta$	Calculation	Error	Corrections		
		x_1	x_2	Y_d	w_1	w_2	θ		Y	e	Δw_1	Δw_2	$\Delta \theta$
1	1	0	0	0	0.1	-0.1	0.3	-0.3	0	0	0	0	0
	2	0	1	0	0.1	-0.1	0.3	-0.4	0	0	0	0	0
	3	1	0	0	0.1	-0.1	0.3	-0.2	0	0	0	0	0
	4	1	1	1	0.1	-0.1	0.3	-0.3	0	1	0.1	0.1	-0.1
2	5	0	0	0	0.2	0	0.2	-0.2	0	0	0	0	0
	6	0	1	0	0.2	0	0.2	-0.2	0	0	0	0	0
	7	1	0	0	0.2	0	0.2	0	1	-1	-0.1	0	0.1
	8	1	1	1	0.1	0	0.3	-0.2	0	1	0.1	0.1	-0.1
3	9	0	0	0	0.2	0.1	0.2	-0.2	0	0	0	0	0
	10	0	1	0	0.2	0.1	0.2	-0.1	0	0	0	0	0
	11	1	0	0	0.2	0.1	0.2	0	1	-1	-0.1	0	0.1
	12	1	1	1	0.1	0.1	0.3	-0.1	0	1	0.1	0.1	-0.1
4	13	0	0	0	0.2	0.2	0.2	-0.2	0	0	0	0	0
	14	0	1	0	0.2	0.2	0.2	0	1	-1	0	-0.1	0.1
	15	1	0	0	0.2	0.1	0.3	-0.1	0	0	0	0	0
	16	1	1	1	0.2	0.1	0.3	0	1	0	0	0	0
5	17	0	0	0	0.2	0.1	0.3	-0.3	0	0	0	0	0
	18	0	1	0	0.2	0.1	0.3	-0.2	0	0	0	0	0
	19	1	0	0	0.2	0.1	0.3	-0.1	0	0	0	0	0
	20	1	1	1	0.2	0.1	0.3	0	1	0	0	0	0

Example 2

Learn the **OR** logical operation using a perceptron.

Inputs		OR
x_1	x_2	$x_1 \cup x_2$
0	0	0
0	1	1
1	0	1
1	1	1

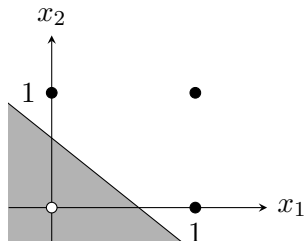


Can we learn **XOR** (exclusive OR) logical operation with the same setup?

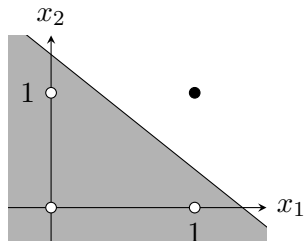
Inputs		XOR
x_1	x_2	$(x_1 \cap \overline{x_2}) \cup (\overline{x_1} \cap x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

XOR?

OR



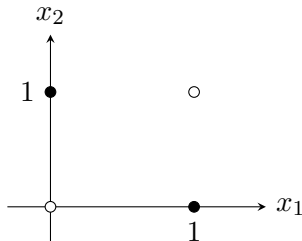
AND



How about XOR?

What is the two dimensional plot for an exclusive OR operation?

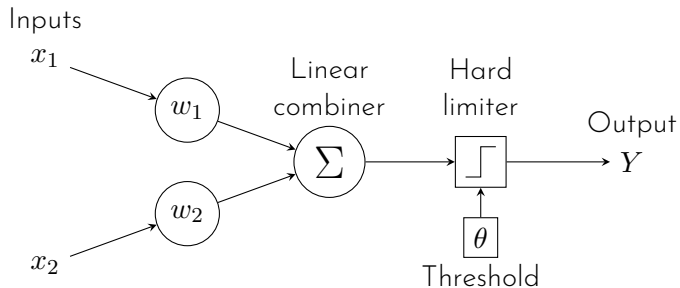
XOR?



Is this a linearly separable function?

i.e. can we draw a straight line to separate the two classes?

Activation function in a perceptron



What is the activation function in a perceptron?

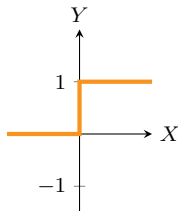
The hard limiter, in the case of learning AND and OR operations, this is

$$Y = \begin{cases} 1, & X \geq \theta \\ 0, & X < \theta \end{cases}$$

What are other activation functions?

The common activation functions are

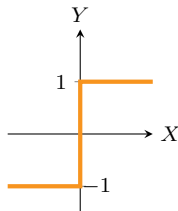
Step function



$$Y = \begin{cases} 1, X \geq 0 \\ 0, X < 0 \end{cases}$$

hard limit function

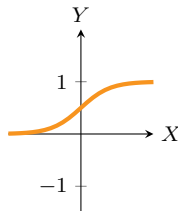
Sign function



$$Y = \begin{cases} +1, X \geq 0 \\ -1, X < 0 \end{cases}$$

hard limit function

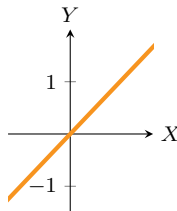
Sigmoid function



$$Y = \frac{1}{1+e^{-X}}$$

output will always be in
the range between 0 and
1 for all inputs between
 $\pm\infty$

Linear function



$$Y = X$$

often used for linear
approximation

If we use sigmoid or linear function instead of the step function, will a perceptron be able to learn an XOR operation?

No. Regardless of hard-limit or soft-limit activation function, single-layer perceptrons make decisions in the same way.

Single-layer perceptrons can only be used to classify linearly separable patterns.

So?

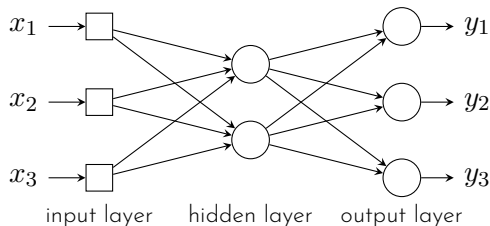
Then, how do we solve problems that are not linearly separable?

Multilayer neural networks.

Multilayer neural networks

A multilayer perceptron is a feedforward neural network with one or more hidden layers.

The basic structure consists of



Why is hidden layer 'hidden'?

- ▶ The hidden layer is sheltered from direct interaction with the input and output signals.
- ▶ Neurons in the hidden layer cannot be observed through the input/output behaviour of the network.
- ▶ There is no obvious way to know what the desired output of the hidden layer should be.
- ▶ The desired output of the hidden layer is determined by the layer itself.

SUNWAY UNIVERSITY Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

Why do we need a hidden layer?

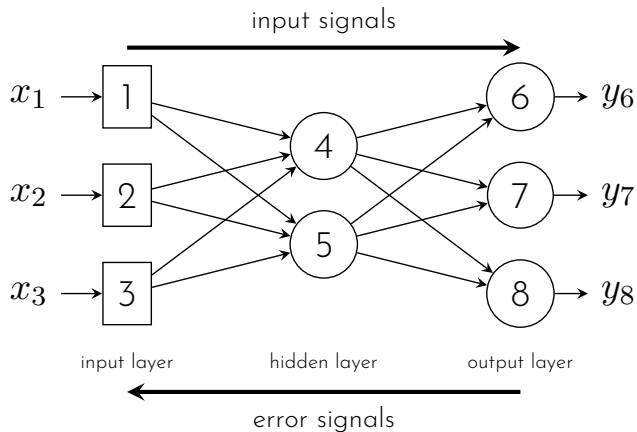
- ▶ The weights of the hidden neurons represent the hidden features in the input patterns.
- ▶ With one hidden layer $\rightarrow$ continuous functions
- ▶ With two hidden layers $\rightarrow$ continuous + discontinuous functions
- ▶ Can a multilayer neural network has more than 2 hidden layers?

How do multilayer neural networks learn?

One popular method: back-propagation algorithm

- ▶ similar to that of a perceptron
 1. training input is presented to the network input layer,
 2. the network propagates the input from input layer to hidden layer and then to the output layer before generating the output
 3. if the generated output is different from the desired output, an error is calculated and propagated backwards from the output layer to the input layer
 4. the weights are adjusted as the error is propagated
- ▶ typically the sigmoid function is used as the activation function

Back-propagation training algorithm



Back-propagation training algorithm

Set all weights, w and threshold θ to random numbers in range $[-0.5, 0.5]$.

Repeat the process of

Activation - Weight training - Iteration

until termination (convergence)



Calculate the output of hidden neurons

$$y_j = \text{sigmoid} \left[\sum_{i=1}^n w_{ij} x_i - \theta_j \right]$$

Calculate the output of output neurons

$$y_k = \text{sigmoid} \left[\sum_{j=1}^n w_{jk} x_j - \theta_k \right]$$

Calculate error

$$e_k = y_{d,k} - y_k$$

Calculate error gradient for
output neurons

$$\delta_k = y_k(1 - y_k)e_k$$

hidden neurons

$$\delta_j = y_j(1 - y_j) \sum_{k=1}^l \delta_k w_{jk}$$

Calculate weight corrections for

output neurons

$$\Delta w_{jk} = \alpha \times y_j \times \delta_k$$

hidden neurons

$$\Delta w_{ij} = \alpha \times x_i \times \delta_j$$

Calculate threshold corrections $\Delta \theta_m = \alpha \times (-1) \times \delta_m$

Calculate new threshold

$$\theta_m = \theta_m + \Delta \theta_m$$

Calculate new weight for

output neurons

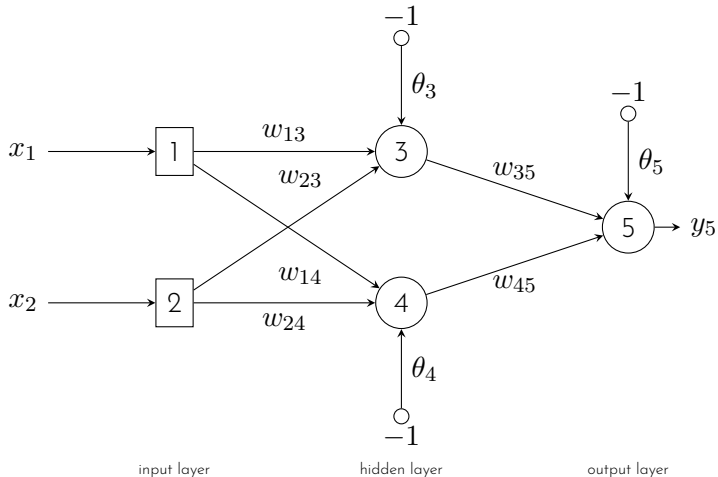
$$w_{jk} = w_{jk} + \Delta w_{jk}$$

hidden neurons

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

Can we learn **XOR** (exclusive OR) logical operation with a multilayer neural network?

Inputs		XOR
x_1	x_2	$x_1 \cup x_2$
0	0	0
0	1	1
1	0	1
1	1	0



How do we terminate the process?

When sum of squared errors is less than a threshold (for e.g. 0.001).

If we train the network with different initial values (weights and thresholds), will we find the same solution or a different one?



SUNWAY
UNIVERSITY

Thank you

CSC3034 Current Course Update

06 Artificial Neural Networks - handout

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current Lab 6 trains and evaluates a scikit-learn MLP on the Wine dataset.
- Current Lab 7 visualizes neural-network hyperplanes on standardized Iris features.
- Current Lab 8 adds Keras CNN/LSTM examples and Isaac Sim vision demonstrations.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.